Google Summer of Code 2026
Machine Learning for Science (ML4SCI)

GSoC 2026

Project Proposal

By

Sourish Dutta

On

Data Processing Pipeline for the LSST (DeepLense)

Google Summer of Code 2026 | Project Proposal

Data Processing Pipeline for the LSST

DeepLense | ML4SCI

Submitted by

Sourish Kumar Dutta

duttasourish12@gmail.com | +91 90387 31606

GitHub | LinkedIn | Kaggle

IIT (ISM) Dhanbad | B.Tech (2025–2029)

CyberLabs - ML Division | IST (UTC+5:30)

Mentors

Michael Toomey (MIT) | Sergei Gleyzer (U. Alabama)

Anna Parul (Observatoire de Paris) | Lucca Paris (Brown U.)

Table of Contents

About Me **4**
Student Information 4
University Information 4
Background Work and Programming Skills **4**
Evaluation Tasks **6**
Common Test I - Multi-Class Substructure Classification 6
Understanding the Data 6
My Approach and Implementation 7
Results 7
Specific Test V - Gravitational Lens Finding 9
Understanding the Data 9
My Approach and Implementation 9
Results 9
Project Information **12**
Abstract 12
Ideas **13**
The LSST Data Challenge 13
Why is this Problem Non-Trivial? 13
Project Plan / Approach **14**
1. Butler Integration Layer 14
2. Preprocessing Pipeline 14
3. DeepLense Task Adapters 15
4. End-to-End Testing on DESC DC2 and HSC - SSP 16
Timeline (Tentative) **17**
End-Term Evaluation & Wrap-up: 19
Deliverables **19**
Future Scope **20**
Motivation for GSoC **20**
Achievements **21**
Hackathons and Competitions 21
Relevant Projects 22
Certifications 22

About Me

Student Information

Name	Sourish Kumar Dutta
Email	duttasourish12@gmail.com
Phone	+91 90387 31606
Timezone	IST (UTC+5:30), Dhanbad, India
GitHub	github.com/sourishkumardutta
LinkedIn	linkedin.com/in/sourishkumardutta
Kaggle	kaggle.com/sourishkumardutta
Address	Kolkata – 700048, West Bengal

University Information

University	Indian Institute of Technology (ISM) Dhanbad
Department	Petroleum Engineering
Degree	B.Tech
Batch	2025 - 2029
ML Club	CyberLabs - ML Division (Member since January 2026)

Background Work and Programming Skills

I study at IIT (ISM) Dhanbad, one of India's top technical institutes. My engineering coursework focuses on applied physics, including reservoir simulation, seismic inversion, and subsurface data analysis. These problems share similar methods with gravitational lensing. They involve a physical forward model, noisy high-dimensional observations, and the goal of inverting or classifying them. This approach feels natural to me.

Before university, I was already creating ML systems from scratch. I built a scalar autograd engine with a dynamic computation graph and manual backpropagation. I also developed a character-level language model with batch normalization and Kaiming initialization, as well as feedforward networks in NumPy. I joined IIT ISM's one of the best coding club, CyberLabs, because I was already engaged in this work.

My Task V result demonstrates my research readiness. I achieved a test AUC of 0.9502 on real observational lens-finding data. I did this by accurately diagnosing and addressing a 100:1 class imbalance meticulously.

- **Languages:** I am proficient with Python (primary), C/C++ for algorithms and systems work
- **PyTorch:** custom training loops, mixed precision (torch.cuda.amp), DataLoaders, WeightedRandomSampler, checkpoint management, AMP API migration
- **Deep Learning architectures:** CNNs including ResNet18, EfficientNet-B0/B3, MobileNetV3; two-phase fine-tuning; ImageNet transfer learning
- **Training infrastructure:** Kaggle dual T4 GPU environments, resume-from-checkpoint workflows, OOM debugging
- **Scientific Python:** NumPy, Pandas, Matplotlib, Scikit-learn, ROC/AUC evaluation, class imbalance metrics
- **EEG and signal processing:** MNE-Python, CEBRA, ICA, pyprep (used for the NeuroDyads GSoC pre-task)
- **Other tools:** Flask, Next.js, FastAPI, QuTiP, genetic algorithms, TF-IDF retrieval

Evaluation Tasks

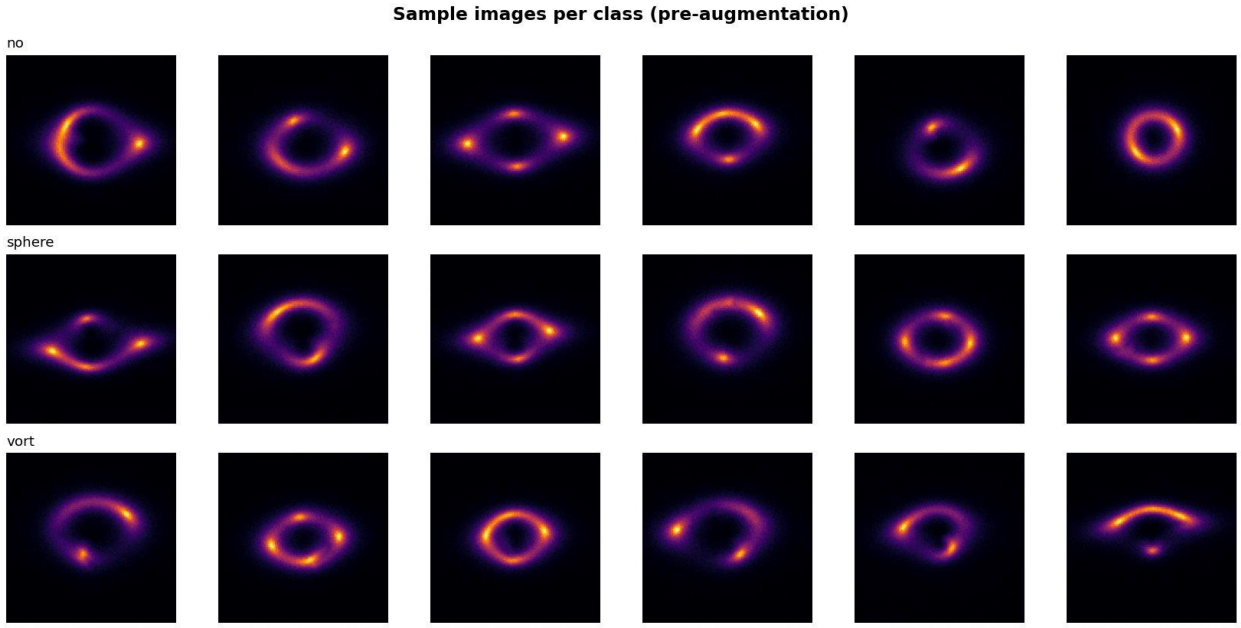
I completed both the mandatory Common Test I (multi-class classification) and Specific Test V (gravitational lens finding), which is the required test for this project track. Below I describe my approach, implementation choices, and results for each.

Common Test I - Multi-Class Substructure Classification

Understanding the Data

The task is three-class classification of simulated strong lensing images: no substructure, subhalo substructure and vortex substructure (a signature of superfluid dark matter condensates). Images are single-channel, 150×150 pixels, normalised using min-max scaling. The dataset has mild class imbalance - the sphere class is underrepresented, which causes naive classifiers to under-recall it.

The scientific context comes directly from Alexander et al. (2019): different dark matter models produce morphologically distinct signatures in strong lensing images, and CNNs can distinguish these more reliably than traditional statistical methods. The vortex class is particularly compelling - detecting it would be direct evidence for superfluid dark matter, outside the standard WIMP paradigm.



The Given Dataset

My Approach and Implementation

Model: I used EfficientNet-B0 with ImageNet-pretrained weights. I adapted the first convolutional layer to accept single-channel input by averaging the pretrained RGB weights across the channel dimension. I replaced the classifier head with Linear (1280 → 3) for three-class output.

Training strategy: I used a two-phase approach. In Phase 1 (3 epochs) I froze all backbone layers and trained only the classifier head (LR = 1e-3) to initialise it before back-propagating into the backbone. In Phase 2 (12 epochs) I unfroze everything and trained with LR = 1e-4. This prevents destroying the pretrained feature representations before the head has learned a reasonable mapping.

Class imbalance: I used WeightedRandomSampler to ensure balanced batches. I also tested focal loss and class-weighted cross-entropy; the combination of WeightedRandomSampler with standard cross-entropy gave the best macro-AUC in practice.

Augmentation: Random horizontal and vertical flips, random rotation (±30°), and random affine jitter - restricted to physically invariant transformations.

Optimisation: AdamW (weight_decay=1e-4), cosine annealing LR schedule, mixed precision. Best checkpoint saved by validation macro-AUC, not loss.

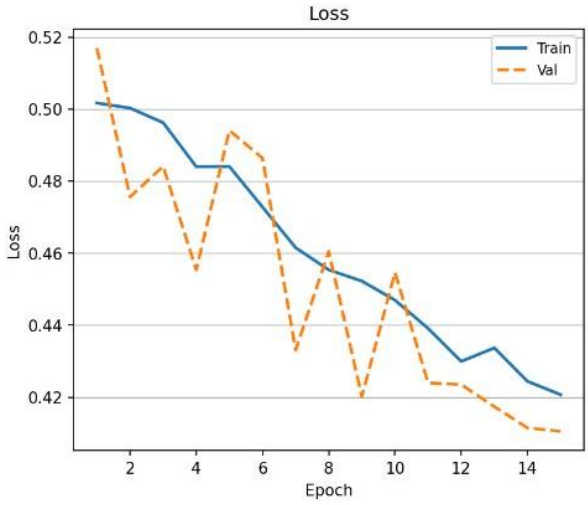
Results

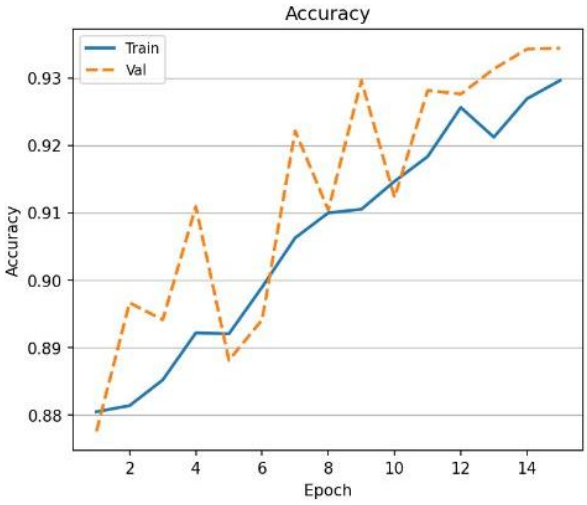
Metric	Value
Macro AUC (best checkpoint, epoch 15)	**0.9893**
Overall Accuracy	**93.44%**
AUC - No Substructure class	**0.993**
AUC - Sphere Substructure class	**0.982**
AUC - Vortex Substructure class	**0.993**
Sphere class recall (confusion matrix)	**86% (2162 / 2500)**
Vortex class recall (confusion matrix)	**95% (2375 / 2500)**

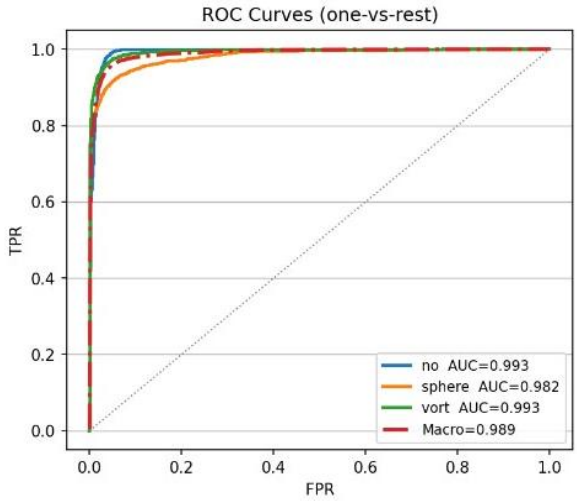
Key observation: The sphere class had the lowest recall throughout training (86%). This is physically expected - spherical subhalos produce diffuse, extended lensing signatures that partially overlap in feature space with the no-substructure class. Alexander et al. (2019) also reported the sphere class as the hardest to classify in their ResNet benchmarks.

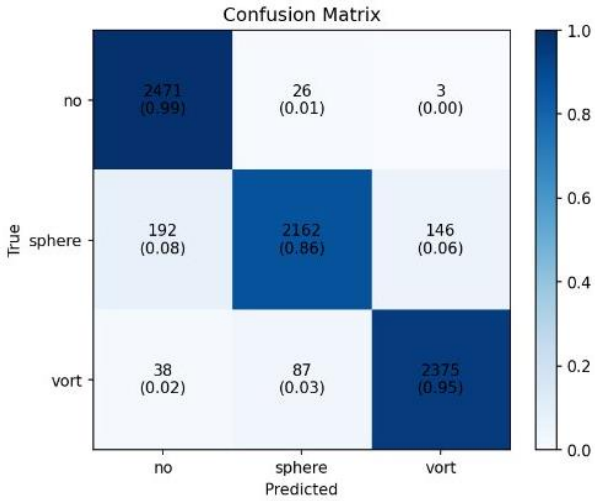
Code: https://github.com/sourishkumardutta/GSoC_DeepLense_PreTask/tree/main/Common_Task

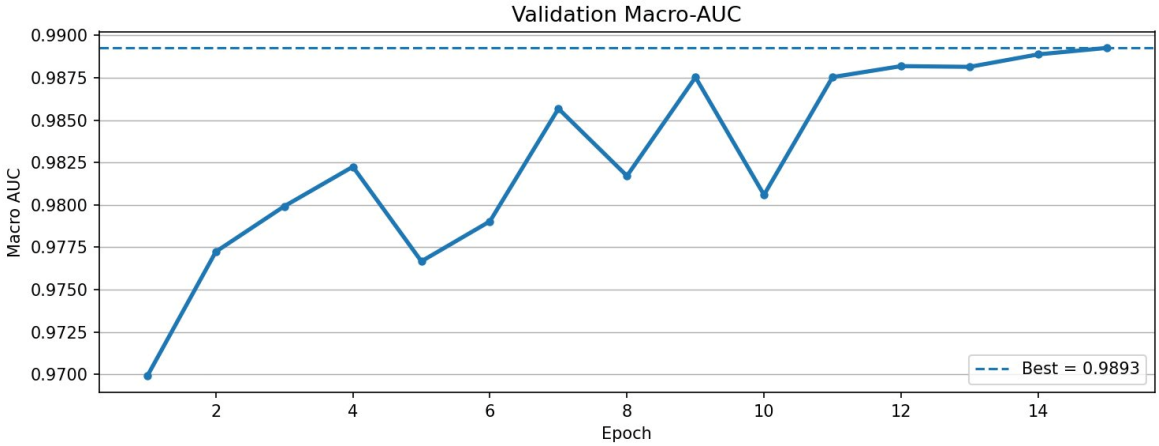
Plots


Loss
Train
Val
Loss
Epoch


Accuracy
Train
Val
Accuracy
Epoch


ROC Curves (one-vs-rest)
no AUC=0.993
sphere AUC=0.982
vort AUC=0.993
Macro=0.989
TPR
FPR


Confusion Matrix
no
sphere
vort
2471 (0.99)
26 (0.01)
3 (0.00)
192 (0.08)
2162 (0.86)
146 (0.06)
38 (0.02)
87 (0.03)
2375 (0.95)
True
Predicted


Validation Macro-AUC
Best = 0.9893
Macro AUC
Epoch

Specific Test V - Gravitational Lens Finding

Understanding the Data

This task uses real observational data - galaxy images structured as train_lenses / train_nonlenses and test_lenses / test_nonlenses. Each object is a 3-channel (g, r, i filter) image of shape (3, 64, 64). The defining challenge is severe class imbalance: roughly 100× more non-lenses than lenses. This is not just a machine learning nuisance - it reflects the fundamental reality of automated lens searches. In real surveys, lenses occur at roughly 1 per 1,000 - 10,000 objects, so even a 1% false positive rate generates an overwhelming number of false positives that human reviewers cannot practically process. Understanding this drove every choice I made in my solution.

My Approach and Implementation

Model: ResNet18 pretrained on ImageNet. The three input channels map naturally to the three survey filters (g, r, i). I replaced the final fully connected layer with Linear (512 → 1) for binary output.

Loss function: BCEWithLogitsLoss with a positive class weight set to the empirical non-lens to lens ratio (approximately 100). This directly penalises false negatives more heavily during training - appropriate given the scientific priority of not missing real lenses.

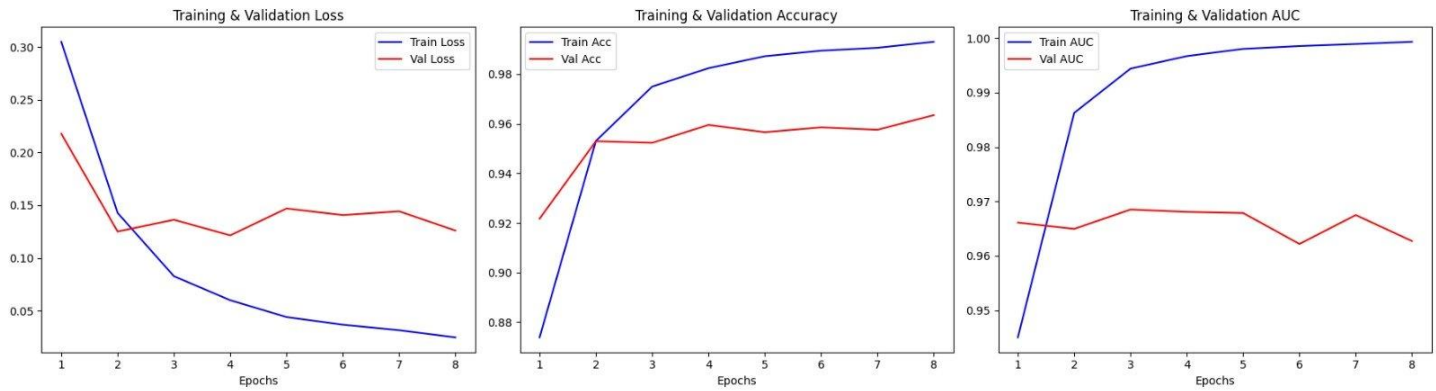
Sampling: WeightedRandomSampler to create roughly balanced batches, combined with augmentation (random flips, rotations, crops) applied specifically to the minority lens class.

Threshold selection: I chose the decision threshold on the validation set to maximise F1, rather than using the default 0.5. In highly imbalanced settings the default threshold is almost never optimal.

Training: 8 epochs, AdamW optimiser, cosine annealing schedule, mixed precision.

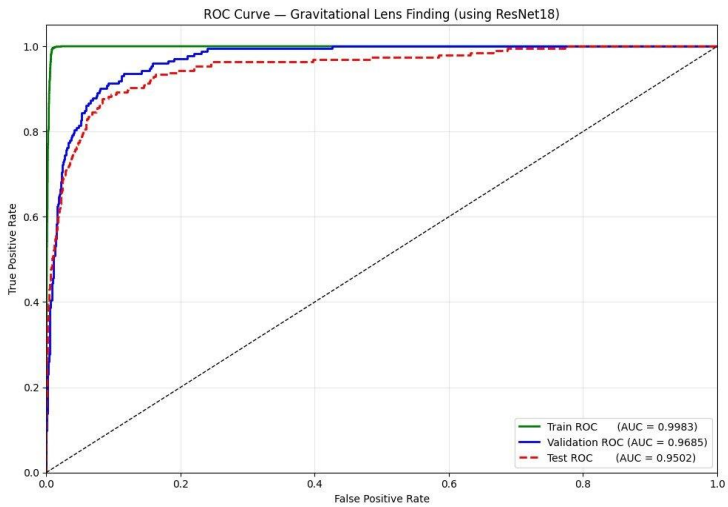
Results

Metric	Value
Train AUC	**0.9983**
Validation AUC	**0.9685**
Test AUC	**0.9502**
True Positives (lenses correctly identified)	**143 / 195 (73.3%)**
False Positives (non-lenses above threshold)	**692 / 19,455 (3.6%)**
True Negatives (non-lenses correctly rejected)	**18,763 / 19,455 (96.4%)**

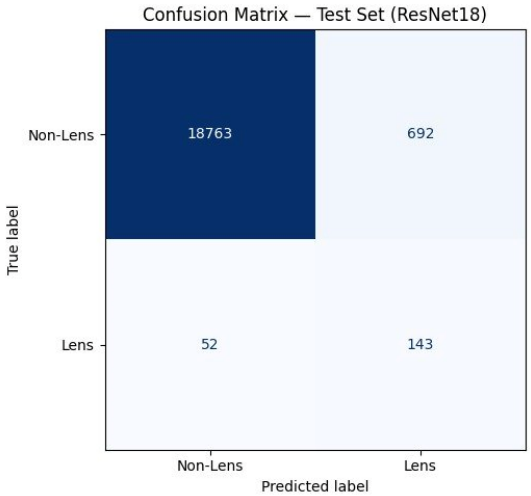


ResNet18 training curves for Task V: loss, accuracy, and AUC across 8 epochs.

Validation AUC stabilises around 0.965–0.970 from epoch 4 onward.



Train AUC = 0.9983, Validation AUC = 0.9685, Test AUC = 0.9502.



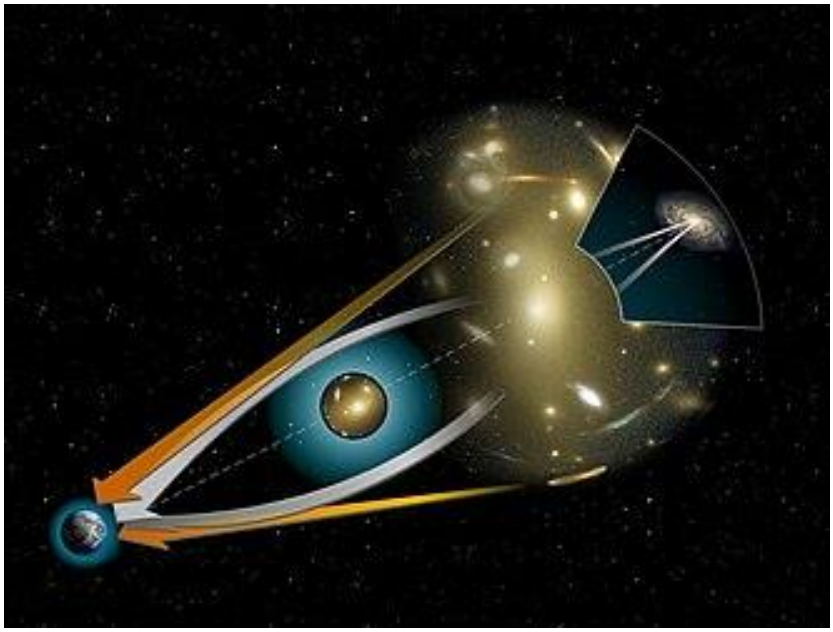
143 of 195 lenses correctly identified, 692 false positives out of 19,455 non-lenses.

Analysis:

The drop from train AUC (0.9983) to test AUC (0.9502) is the central diagnosis of this project, not a failure. It reflects the domain gap between what the model learns on a limited training sample and what it encounters across the full test set.

For context: Parul et al. (2024) showed that a naive simulation-trained ENN classifier drops from AUROC 0.995 on simulated data to 0.921 on real HSC observations. My **ResNet18 baseline at 0.**9502 - without any domain adaptation - **already exceeds their unadapted real-data baseline.** That gives me a solid foundation for the domain adaptation work I plan to do during GSoC.

Code: https://github.com/sourishkumardutta/GSoC_DeepLense_PreTask/tree/main/Specific_Test_V



This is how Lenses are formed due to the apparent bending of Light.

Project Information

Field	Details
Organisation	ML4SCI
Project Title	Data Processing Pipeline for the LSST (DeepLense)
Sub-project	DeepLense
Duration	175 hours
Difficulty	Intermediate / Advanced
Mentor 1	Michael Toomey – Massachusetts Institute of Technology
Mentor 2	Sergei Gleyzer – University of Alabama
Mentor 3	Anna Parul – Observatoire de Paris
Mentor 4	Lucca Paris – Brown University
Reference Papers	Alexander et al. (2020) arXiv:2008.12731; Alexander et al. (2019) arXiv:1909.07346

Abstract

The Vera C. Rubin Observatory's Legacy Survey of Space and Time (LSST) will generate about 20 TB of raw data each night. It will create calibrated images of billions of objects over the course of ten years. It is expected to discover around 100,000 strong gravitational lenses, which is vastly more than what we know today. Before this influx of data arrives, we urgently need automated machine learning pipelines that can connect directly with the Rubin Science Pipelines data system.

This project aims to create such a pipeline. It will integrate the LSST Butler data access middleware with DeepLense machine learning workflows. The pipeline will address the entire data lifecycle, starting from raw Butler repository queries, through preprocessing, format standardization, model inference, and result cataloguing.

It will be thoroughly tested from start to finish using Rubin's DESC DC2 mock survey and the existing HSC-SSP lens-finding dataset. The result will be a modular, documented, reusable software framework that allows DeepLense to function at LSST scale right from the start.

Ideas

The LSST Data Challenge

The **Rubin Science Pipelines** (pipelines.lsst.io) provides a sophisticated stack for reducing raw LSST observations into science-ready data products: single-frame processing, coaddition, source measurement, and multi-band catalog analysis.

The central abstraction is the Butler, a data access layer (lsst.daf.butler) that mediates between science code and the underlying data repository. All downstream science - including DeepLense applications - must speak Butler's language.

The current DeepLense ecosystem is **built around offline datasets (the simulated lensing dataset, the HSC-SSP observational dataset).** There is no systematic bridge between the LSST data infrastructure and DeepLense's classification, super-resolution, or lens-finding workflows. When LSST goes online, the absence of this bridge would prevent DeepLense from processing real LSST data without significant ad-hoc adaptation for each use case. This project builds the bridge in advance.

Why is this Problem Non-Trivial?

Integrating the Rubin Pipelines with ML workflows is not simply a matter of format conversion.

The main problems that will be faced are:

- **Butler API complexity:** The Butler uses dataset types, data ID queries, and collections that require explicit schema understanding. DeepLense needs a clean abstraction over this to avoid requiring every contributor to become a Butler expert.

- **Data format heterogeneity:** LSST exposures are FITS-based with WCS metadata, PSF models, mask planes, and variance maps. DeepLense models expect PyTorch tensors of a fixed spatial resolution. The preprocessing bridge must handle multi-band cutout extraction, PSF standardisation, normalisation, and mask-aware quality filtering in a composable pipeline architecture.

- **Scale:** The DESC DC2 mock survey alone covers approximately 300 square degrees with ~10^7 objects. Any pipeline design that is not batch-parallelisable and memory-efficient will fail at this scale.

- **Reproducibility across surveys:** The pipeline must make DeepLense models applicable *not just to LSST, but to HSC-SSP, DES, and KiDS* - surveys with different pixel scales, filter sets, PSF characteristics, and noise properties.

Project Plan / Approach

1. Butler Integration Layer

I will build a Butler Integration Layer: a Python module (deeplense_lsst.butler) that wraps **lsst.daf.butler** to provide a clean interface for DeepLense applications.

This module will expose:

- **DatasetQuery:** Declarative specification of Butler data ID constraints (sky area, filter, visit, tract/patch) that translates to Butler registry queries.

- **CutoutExtractor:** Batch extraction of fixed-size cutouts centred on catalog coordinates from coadd images, leveraging lsst.afw.image for WCS-aware spatial indexing. Output will be NumPy arrays carrying per-pixel mask and variance maps.

- **ExposureMetadata:** Structured access to per-cutout metadata: PSF parameters, airmass, seeing FWHM, filter name, sky background, and photometric calibration zero-point. This metadata is critical for domain adaptation and cross-survey normalisation.

The layers will be tested against the DESC DC2 simulated LSST dataset, which is publicly available in Butler repository format and represents realistic LSST data volume and characteristics.

2. Preprocessing Pipeline

Raw FITS cutouts cannot be fed directly to DeepLense models. I will implement a composable preprocessing pipeline with the following stages:

- **Multi-band stacking:** For the lens-finding task, images in the g, r, i (or u, g, r, i, z, y) filters are stacked into multi-channel tensors. The pipeline will handle variable filter availability gracefully.

- **PSF standardisation:** LSST observations have spatially varying PSFs. I will implement PSF-matched convolution to bring all cutouts to a common PSF FWHM before normalisation, using kernel computation from the Butler-provided PSF model.

- **Normalisation and masking:** Masked pixels (from the LSST mask plane) are replaced by local background estimates rather than set to zero, preventing the CNN from treating mask boundaries as features. Flux normalisation options include per-image min-max and per-band z-score.

- **Quality filtering:** Automatic rejection of cutouts with excessive masking, saturated pixels, or missing data - with configurable quality thresholds.

All preprocessing stages will be implemented as composable transforms compatible with PyTorch's.transforms API, so they slot directly into existing DeepLense DataLoader workflows.

3. DeepLense Task Adapters

Three DeepLense task adapters will be built on top of the preprocessing pipeline:

- **Lens Finding Adapter:** Connects the Butler cutout pipeline to the ResNet18 / EfficientNet-B3 lens classifiers from Test V. Will include inference-time batching, GPU parallelism, and score-file output in FITS catalog format for cross-matching with existing lens catalogues.

- **Multi-Class Classification Adapter:** Connects the preprocessing pipeline to the EfficientNet-B0 Common Test I classifier for substructure type prediction on real survey cutouts of confirmed lenses.

- **Super-Resolution Adapter:** Connects the Butler pipeline to a super-resolution model, enabling upsampling of low-resolution LSST which changes towards the resolution expected by DeepLense downstream classifiers.

Each adapter will have a minimal configuration API so researchers can point it at a Butler repository, specify data selection criteria, and get inference results with a single command.

4. End-to-End Testing on DESC DC2 and HSC - SSP

The completed pipeline will be validated by running the lens-finding adapter on a subset of the DESC DC2 Wide-Fast-Deep footprint and on the HSC-SSP PDR2 dataset used in my Task V submission. Key validation metrics:

- **Consistency:** AUC scores on HSC-SSP must be within 0.5% of the standalone Task V result (confirming the pipeline adds no unintended preprocessing artifacts).
- **Scalability:** Memory footprint and runtime scaling across 10^4, 10^5, and 10^6 objects tested using the DC2 dataset.
- **Cross-survey transfer:** The same pipeline configuration evaluated on both HSC-SSP and DC2 without task-specific tuning, measuring cross-survey AUC degradation.

Timeline (Tentative)

Total project length: 175 hours.

Coding period: approximately 12 weeks at 14–15 hours per week.

Week	Dates	Tasks	CheckPoints
Pre	Until 1 Jun	Study Butler v29 docs and DC2 data schema; reproduce DC2 cutout extraction locally; set up repo structure and CI; dry-run Butler query against DESC DC2 public dataset	
W1	Jun 2–8	Implement DatasetQuery and CutoutExtractor modules against DC2 Butler repo; unit tests for WCS-aware spatial indexing	**Butler layer reads DC2 cutouts completion**
W2	Jun 9–15	Implement ExposureMetadata extraction (PSF, seeing, zero-point); add multi-band stacking transform; integrate mask plane handling	
W3	Jun 16–22	Implement PSF-matched convolution; normalisation transforms (z-score, asinh); quality filtering; torchvision-compatible transform API	**Preprocessing pipeline feature-complete**
W4	Jun 23–29	Build Lens Finding Adapter; port Task V ResNet18 inference to use Butler pipeline; FITS catalog output with score columns	
W5	Jun 30–Jul 6	Build Multi-Class Classification Adapter (EfficientNet-B0 from Common Test I); test on confirmed HSC-SSP lenses	

W6	Jul 7–13	Build Super-Resolution Adapter; integrate single command-line interface for all three adapters	**All three task adapters functional**
Midterm	~Jul 14–20	Progress review with mentors; adjust plan if needed	**Midterm evaluation**
W7	Jul 21–27	End-to-end run on DESC DC2 footprint subset (10^5 objects); measure throughput and memory footprint	
W8	Jul 28–Aug 3	Consistency validation: compare HSC-SSP lens-finding AUC via pipeline vs standalone Task V baseline; confirm <0.5% deviation	**Validation passed on HSC-SSP**
W9	Aug 4–10	Scalability testing: 10^4 / 10^5 / 10^6 object runs on DC2; profile and optimise bottlenecks (I/O, CPU preprocessing, GPU batching)	
W10	Aug 11–17	Cross-survey evaluation: apply HSC-configured pipeline to DC2 and measure cross-survey AUC degradation without retuning	**Cross-survey report complete**
W11	Aug 18–24	Documentation: comprehensive README, Sphinx API docs, Jupyter demo notebooks for each pipeline stage; packaging with pip-installable setup	
W12	Aug 25–31	Final code cleanup; blog post for ML4SCI website; repository polish; final GSoC evaluation submission	**Final evaluation**

End-Term Evaluation & Wrap-up:

By the end of the program, I will ensure all implementations are thoroughly documented, tested, and accessible in the project repository. I aim to provide clear examples and instructions for reproducing the results.

I plan to actively communicate progress weekly with my mentors and incorporate their feedback throughout the project. The buffer week is included to manage any potential blockers effectively.

Following GSoC, I am interested in contributing further, potentially through blog posts explaining the work or contributing to any resulting research papers

Deliverables

Upon completion of this GSoC project, the following deliverables will be provided:

- A pip-installable Python package (deeplense_lsst) with full Butler integration layer, composable preprocessing transforms, and three task adapters (lens finding, multi-class classification, super-resolution).
- Jupyter notebooks demonstrating end-to-end workflows: Butler query → cutout extraction → preprocessing → DeepLense inference → catalog output.
- Benchmark report comparing throughput, memory footprint, and AUC consistency between standalone and pipeline-integrated inference on DESC DC2 and HSC-SSP.
- Cross-survey evaluation results quantifying pipeline generalisation from HSC-SSP to DC2.
- Comprehensive documentation: installation guide, API reference, configuration examples, and a mock survey demo using DESC DC2 public data.
- Short blog post for the ML4SCI website summarising the project and demonstrating the pipeline on a concrete example.

Future Scope

The ability to create AI models that adapt to changing data, explored in this project, is crucial well beyond physics. Techniques for continuous learning and evaluating model robustness are vital for making AI more reliable in many real-world applications.

The most natural extension is alert stream integration: once **LSST** begins operations, the **APDB** (Alert Production Database, lsst.dax.apdb) will emit real-time difference imaging alerts for transient events. A natural follow-on project would adapt the lens-finding pipeline to operate on alert stream data, enabling real-time strong lens discovery from nightly LSST observations.

The composable preprocessing architecture also generalises to other ML4SCI applications beyond lensing: **CMB map processing**, galaxy morphology classification, and photometric redshift estimation from Rubin multi-band photometry are all potential users of the Butler integration layer.

Motivation for GSoC

I got into machine learning by building everything from scratch. My first serious project was a **scalar autograd engine** - implementing the **Value class, the dynamic computation graph, and backpropagation by hand** - and then **validating every gradient against PyTorch.** It was slow, sometimes frustrating, and completely worth it. I have a genuine feel for what's happening inside these models because I built the machinery before I used the libraries.

I understand not just what the tools do but why they are structured as they are, which matters a lot when debugging edge cases in a complex multi-component system like this pipeline. What draws me specifically to this project is that it is not a trivial problem.

The gap between the **LSST data ecosystem** and the **DeepLense ML ecosystem** is real and blocking. Closing it is not glamorous, but it will enable all the downstream to actually run on **LSST** data. That kind of foundational contribution to scientific infrastructure is exactly what I want to do.

Working with **Michael Toomey and Sergei Gleyzer**, who wrote both reference papers for this project, and **Anna Parul,** whose domain adaptation results directly motivate the cross-survey evaluation component, I think is an amazing chance to learn from the people who created the scientific context which this pipeline uses.

Achievements

Hackathons and Competitions

1. **1st Place - ML Track, PhysisTechne Symposium Hackathon, IIT (ISM) Dhanbad**

I built QSetup - an autonomous AI for designing photonic quantum circuits. The system uses a genetic algorithm (selection, crossover, mutation, elitism) with quantum state fidelity scoring via QuTiP simulation, and TF-IDF retrieval over research papers to ground generated designs in the literature. Delivered as a full-stack Flask + Next.js application with interactive visualisation.

2. **2nd Place - AI Summit 2026 (60–80 participating teams)**

Autonomous job-search and auto-apply agent: the system ingests a resume, GitHub, and LinkedIn profile; semantically matches them to job listings; generates grounded cover letters via the Gemini API; and auto-submits with safety gates. Team project - I contributed to AI design, prompt engineering, and system architecture. Built with FastAPI and Next.js.

3. **3rd Place - CyberLabs WOC Kaggle Competition (Private Leaderboard, score 0.66725)**

Binary classification on a 3,072-feature dataset with 10% class imbalance. I iterated through 15+ approaches including PCA (95% variance threshold), stratified K-fold out-of-fold cross-validation, decision threshold optimisation (balanced accuracy 0.52 → 0.78), and probability calibration. Key finding: calibrated logistic regression with PCA preprocessing outperformed deep MLPs under strict cross-validation.

4. **Currently competing - LC × MNNIT AI Hackathon on Kaggle**

I am actively competing in this live Kaggle hackathon alongside the GSoC submission period.

5. **AIR 139 - WBJEE | 95% CBSE Class 12 | 96% CBSE Class 10**

Relevant Projects

- **Micrograd + Character-Level Language Model:**

  Scalar autograd engine from scratch with dynamic compute graph and manual backpropagation, validated against PyTorch. Extended to a character-level name generator MLP with learned 2D embeddings, batch normalisation, Kaiming initialisation, and 100k-step mini-batch SGD.

- **ML Library from Scratch (CyberLabs WOC):**

  Linear, Polynomial, and Logistic Regression; KNN (macro-F1 0.742); K-Means; feedforward networks (96.3% validation accuracy) - all in NumPy.

- **EEG Hyperscanning Analysis (NeuroDyads GSoC pre-task):**

  Preprocessed simultaneous 64-channel dyadic EEG (bandpass filtering, bad-channel interpolation via pyprep, ICA artifact removal). Trained CEBRA on joint speaker-listener embeddings; achieved 98.35% KNN decoding accuracy versus 49.45% shuffled baseline.

Certifications

- Convolutional Neural Networks - DeepLearning.AI / Andrew Ng (March 2026)
- Machine Learning Specialization - DeepLearning.AI / Stanford Online (December 2025)

Thank You